

Chronic Wound Classification Using Gated Multi-Backbone Fusion

A leakage-free deep learning pipeline with a learned gating network — baseline and improvement round, fully audited and reproducibly documented.



79.28% FINAL ACCURACY	79.43% MACRO F1	n = 111 CLEAN TEST SET
PASS LEAKAGE AUDIT		

AUTHOR	Abdur Rehman Khan · 24K-0767 · BCS-G
SUPERVISOR	Ms. Sania Urooj, Department of Artificial Intelligence
INSTITUTION	FAST National University of Computer and Emerging Sciences (NUCES), Karachi
EXTENSION OF	"A Decision-Level Fusion Framework for Enhanced Multi-Class Chronic Wound Detection" — Shamoon, Mustafa, Urooj, Mushtaq (FAST NUCES)
DATASET	AZH Chronic Wound Dataset · Six-Class ROI Classification
DATE	September 2026

Contents

1. Executive Summary	3
2. Evidence Levels Used in This Report	4
3. The Classification Task	5
4. System Architecture	6
5. The Leakage Incident (Debugging Record)	7
6. What Was Rebuilt	8
7. Dataset and Split Methodology	9
8. Training Configuration	11
9. Results on the Clean Test Set	13
10. Interpretation	15
11. Statistical Uncertainty	16
12. Verification Artifacts	17
13. Limitations	19
14. Conclusions	20
15. Future Work (priority order)	21
16. Improvement Round (Second Iteration)	22
Appendix A. Reference Command Sequence (verified against README history)	25
Appendix B. Plain Language Glossary	26
Appendix C. Status Scoreboard	27

1. Executive Summary

Relationship to prior work. This report documents an extension of a previously published study, "*A Decision-Level Fusion Framework for Enhanced Multi-Class Chronic Wound Detection*" by Shamoon, Mustafa, Urooj and Mushtaq (FAST NUCES), which combined VGG19, DenseNet201, MobileNetV2 and NASNet-Mobile with decision-level fusion on the AZH wound dataset. This extension replaces that study's fusion stage with a learned gating network trained on out of fold predictions, and re-runs the entire evaluation under a strict, audited, leakage-safe protocol. The published study is cited as prior work throughout; all results reported here were produced independently by this extension's pipeline.

This project classifies a wound photograph into one of six categories: background, diabetic, normal-skin, pressure, surgical or venous. Three pretrained CNN backbones each examine the image, and a small learned gating network combines their probability outputs into a final decision.

During verification, the team found that the earlier evaluation pipeline contained data leakage: a fallback path could copy training images into the test folder, the gating stage reconstructed test paths from training paths, and the old split was not group aware. The earlier high numbers were therefore inflated and were excluded from reporting.

The pipeline was rebuilt from the raw dataset. The corrected system was first evaluated on a held out test set of 111 images (baseline: 73.87% accuracy, 74.49% macro F1). An improvement round was then executed — longer training, Phase 2 fine tuning and label smoothing — with the decision to evaluate made on out of fold metrics only, after which the improved system achieved **79.28% accuracy and 79.43% macro F1** on the same clean test set (Section 16). In the improved configuration the learned gate outperforms every alternative: the best single backbone (+2.98 macro F1), simple averaging (+1.72) and majority voting (+1.76).

The previously ambiguous Phase 2 question is resolved: Phase 2 was not part of the baseline results (Section 8.1) and was subsequently executed as part of the Improvement Round (Section 16).

2. Evidence Levels Used in This Report

Mark	Meaning
Verified (repo)	Confirmed by reading repository code, saved results file, or documented commands at commit <code>92ca93c</code> (2026-09-12).
Verified (arithmetic)	Recalculated independently from the saved confusion matrix.
Verified (local run)	Produced by running the repository audit and analysis scripts on the machine holding <code>data/processed/</code> and <code>outputs/</code> ; outputs captured and pasted into this report.

Everything in this report is in one of these three categories. All verification items are complete.

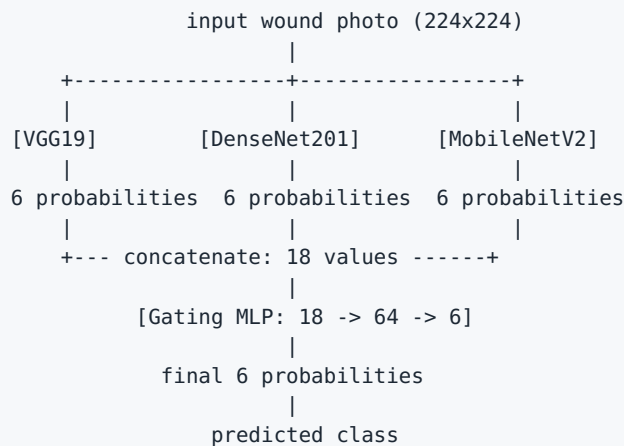
3. The Classification Task

Class	Meaning	Type
background	No wound visible (surface, cloth, background skin)	Non wound
normal-skin	Healthy intact skin	Non wound
diabetic	Diabetic foot ulcer	Wound
venous	Venous leg ulcer	Wound
pressure	Pressure ulcer (bed sore)	Wound
surgical	Surgical wound	Wound

Class to index mapping (alphabetical, verified in code): 0 = background, 1 = diabetic, 2 = normal-skin, 3 = pressure, 4 = surgical, 5 = venous.

The task is hard because the four wound types look similar in photographs. Clinicians separate them largely by body location and patient history, which a cropped ROI photograph may not contain.

4. System Architecture



Components and where they live:

Component	File	Verified (repo)
Backbone builders	<code>scripts/gen_oof_preds.py (build_model)</code>	Yes
Group aware OOF training	<code>scripts/gen_oof_preds.py</code>	Yes
Gating MLP and test evaluation	<code>scripts/fusion/train_gating_mlp.py</code>	Yes
Final metrics report	<code>scripts/evaluate_gated_fusion.py</code>	Yes
Split rebuild	<code>scripts/rebuild_clean_split.py</code>	Yes
Audits	<code>leakage_audit.py , scripts/</code> <code>check_clean_split.py</code>	Yes, outputs captured (Section 12.2)
Ablation table	<code>scripts/make_ablation_table.py</code>	Yes, output captured (Section 12.1)

Notes:

- The gate used in the reported pipeline is `GatingMLP` inside `train_gating_mlp.py` (18 to 64 to 6, Dropout 0.2). The class in `src/models/gating.py` is an older feature based variant that the reported pipeline does not use.
- `configs/default.yaml` is a stale prototype configuration (1r 0.001, 5 epochs, 50 images per class subset, CPU). It does not describe the reported run and is retained as legacy.
- Training input: 224 by 224, ImageNet normalization (mean 0.485/0.456/0.406, std 0.229/0.224/0.225).

5. The Leakage Incident (Debugging Record)

Three separate issues were found and fixed. The remediation is also documented in the repository README history.

5.1 Bug 1: Fallback copy from train into test

The original pipeline could copy a training image into the test directory when the expected test file was missing. This placed already studied material into the exam, so test scores stopped measuring generalization.

5.2 Bug 2: Fabricated test paths in the gating stage

The original gating script reconstructed test paths by replacing `/train/` with `/test/` in OOF metadata and fell back to the same copy behaviour when the mirrored file did not exist. Because augmented names (for example `aug_0000_10.jpg`) never legitimately exist under the test directory, this fallback would fire repeatedly and re-introduce leakage independently of Bug 1.

Fix verified in current code: `train_gating_mlp.py` now builds test rows by scanning `data/processed/test/` directly. A missing test directory raises an explicit error. The code comment states the rule: never reconstruct test paths from train metadata, never copy files across splits.

5.3 Flaw 3: The old split was not group aware

The old cross validation used class balanced splits with no awareness that augmented siblings of one source image existed, so an original and its augmented copies could land on opposite sides of a fold boundary. A filename based duplicate check cannot detect this. The fix is group aware splitting (Section 7.2).

5.4 Why leakage invalidates a result

A leaked model can recognize a specific photograph (skin tone, ruler, shadow) instead of learning what a pressure ulcer looks like. The accuracy number alone cannot distinguish an image recognizer from a disease classifier. Leakage is invisible in practice because it never crashes, always improves the metric, and nobody questions a good number.

5.5 Status of the earlier numbers

All artifacts from the leaked configuration were archived under `archive/cleanup_2026_09_12/` (kept locally, excluded from the repository by design) and are excluded from all reported metrics.

6. What Was Rebuilt

#	Fix	Verified
1	Train/val/test rebuilt from raw ROI source, seed 42, 70/15/15 (<code>re-build_clean_split.py</code>)	Yes (code)
2	Augmentation applied to training only; val and test contain originals only	Yes (code + counts, Section 7.3)
3	<code>StratifiedGroupKFold</code> , group = original source stem (augmentation prefix stripped by regex)	Yes (code)
4	Gating script scans the real test directory, refuses to fabricate	Yes (code)
5	Leaked artifacts archived, excluded from reporting	Yes (README history)
6	Independent audit run: 0 collisions of all three types	Yes (local run, Section 12.2)

Engineering principle: **never fabricate data to satisfy a code path**. A missing file means something upstream is broken; fail loudly instead of inventing the missing piece.

7. Dataset and Split Methodology

7.1 Top level split (verified from `rebuild_clean_split.py`)

- Source: raw AZH ROI folders, one file per source image.
- Seed 42, per class shuffle, then 70% train, 15% val, 15% test.
- Split happens before augmentation, so validation and test contain original images only. No copy fall-back exists in this script.
- Test set: $15 + 23 + 15 + 15 + 19 + 24 = 111$ images (verified from the confusion matrix supports).

7.2 Group aware fold creation (verified from `gen_oof_preds.py`)

`StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=seed)`. Group key = filename stem after stripping the augmentation prefix (`aug_0000_10.jpg` maps to group `10`). All augmented siblings of a source image therefore always stay inside one fold, and each fold's held out side is evaluated with the deterministic transform (resize plus normalize), never the augmentation transform.

Known limitation, stated honestly: stem based grouping removes the augmented sibling route but does not guarantee patient level independence, because two different photographs of the same patient carry different stems. The AZH ROI release does not include patient identifiers, so this limitation is documented rather than solved. The audit's zero group collisions (Section 12.2) confirms the group logic worked as intended across splits.

7.3 Split counts (captured via `scripts/dataset_stats.py`)

Class	Train (after augmentation)	Validation	Test
background	350	15	15
diabetic	350	23	23
normal-skin	350	15	15
pressure	350	15	15
surgical	350	19	19
venous	350	23	24
Total	2100	110	111

Notes: training was augmented to a balanced target of 350 images per class (about 2,100 training files, naming `aug_{index}_{original_stem}{ext}`, seed 42). Validation contains 110 original images and test contains 111 original images; no augmented file exists in either (confirmed by the audit in Section 12.2). Of the 2,100 training files, the originals are the 70% train share of the raw counts (roughly 70 to 112 originals per class), and the remainder up to 350 per class are augmented copies; the exact per class breakdown can be listed with the originals-vs-augmented one liner in `docs/verification_checklist.md`.

7.4 Augmentation (verified from `augment_train_only.py` and training transforms)

- Offline augmentation to roughly 350 images per class (about 2100 training files), seed 42, inside `data/processed/train/` only.
- Additional on the fly augmentation during training: horizontal flip 0.5, vertical flip 0.2, rotation up to 15 degrees.
- Validation, test and all fold held out sides: deterministic resize plus normalize only.

8. Training Configuration

8.1 Backbone training (verified from code and documented commands)

Setting	Value	Evidence
Folds	5, <code>StratifiedGroupKFold(shuffle=True, random_state=42)</code>	Code
Phase 1	Up to 8 epochs, backbone frozen, classification head only	Code + README commands
Phase 2	Not executed for the reported results	See below
Early stopping	Patience 5 on validation loss, best checkpoint kept per fold	Code
Optimizer	Adam	Code
Learning rate	1e-4 (Phase 2 would use lr/10)	Code + README commands
Weight decay	1e-4	Code + README commands
Batch size	16	Code + README commands
Loss	CrossEntropyLoss	Code
Random seed	42 (<code>set_seed</code> , covers random, numpy, torch, cuda)	Code
Pretrained weights	torchvision <code>IMAGENET1K_V1</code> for all three backbones	Code
Hardware	CPU only (no CUDA GPU on the training machine; code selects <code>cuda</code> if available else <code>cpu</code> , and no NVIDIA device exists)	Command history + <code>nvidia-smi</code> absence + timing pattern
Approximate training runtime	VGG19 $\approx$ 24 h; DenseNet201 and MobileNetV2 $\approx$ 3–4 h each; total $\approx$ 30+ h backbone training	Team record
Checkpoints	<code>outputs/01_oof/{model}_clean/checkpoints/{model}_fold{k}_best.pt</code>	Code + README

Phase 2 statement: Three independent records agree. The script's `--phase2` flag is opt-in and defaults to skipping Phase 2; the documented commands for all three backbones do not pass `--phase2` ; and the project notes state Phase 1 only was the policy. Therefore the reported models were trained with **Phase 1 only, up to 8 epochs per fold with early stopping (patience 5)**. Because training progress was printed to console only, the exact number of epochs completed per fold was not persisted; the defensible wording is "up to 8 epochs with early stopping patience 5."

Test time inference protocol: verified as checkpoint averaging. For each backbone, the five fold checkpoints each predict the test set in deterministic order, and the softmax outputs are averaged. The three averaged probability vectors are concatenated into an 18 value input for the gate (function `average_test_probs_for_model`).

8.2 Gating network training (verified from `train_gating_mlp.py`)

Setting	Value	Evidence
Architecture	<code>GatingMLP</code> : Linear(18 to 64), ReLU, Dropout 0.2, Linear(64 to 6)	Code
Input	Concatenated OOF probabilities of the three backbones (18 values)	Code
Internal validation split	20% stratified subset of the OOF data, <code>random_state=42</code>	Code
Validation data source	OOF pool only, not the test set	Code
Max epochs	50	Code
Early stopping	Patience 5 on validation loss	Code
Batch size	64	Code
Learning rate	1e-3	Code
Weight decay	1e-4	Code
Seed	42	Code
Artifacts	<code>gating_mlp_model_v1.pt</code> , <code>gating_test_probs_v1.npy</code> (111 x 18), <code>gating_test_preds_v1.csv</code>	Code + README

Actual stopping epoch was console only and not persisted; the defensible wording is "up to 50 epochs with early stopping patience 5."

8.3 Pipeline order

1. Rebuild clean split from raw ROI data (seed 42)
2. Augment training folder only (seed 42, 350 per class)
3. Audit: `leakage_audit.py` + `check_clean_split.py` -> PASSED (Section 12.2)
4. Per backbone: 5-fold group-aware OOF training, Phase 1 only
5. Train gating MLP on concatenated OOF probabilities
6. Average fold checkpoints per backbone on the clean test set
7. Gate produces final test predictions
8. `evaluate_gated_fusion.py` computes metrics once
9. `make_ablation_table.py` produces the fusion comparison (Section 12.1)

One design note recorded for honesty: the gate is trained on single model OOF probabilities but consumes fold averaged probabilities at test time. Averaging usually makes inputs slightly more confident. This is a common, accepted pattern; it is listed in Section 13 as a limitation rather than hidden.

9. Results on the Clean Test Set

Source: `outputs/03_figures/gated_fusion_evaluation_report.txt` (file present in repository and matching the shared PDF exactly).

9.1 Headline metrics

Metric	Value	Verified
Test images	111	Yes (repo file + matrix)
Correct predictions	82	Yes (arithmetic)
Overall accuracy	73.87%	Yes: $82/111 = 0.73874$
Macro F1	74.49%	Yes: mean of six class F1
Weighted F1	74.33%	Yes: support weighted mean

9.2 Per class results

Terminology note: the results file labels the recall column "per-class accuracy". Those values are **recall** (diagonal divided by row total). The table below uses correct labels.

Class	Precision	Recall	F1	Test images
background	100.00%	93.33%	96.55%	15
diabetic	76.19%	69.57%	72.73%	23
normal-skin	93.33%	93.33%	93.33%	15
pressure	36.84%	46.67%	41.18%	15
surgical	73.33%	57.89%	64.71%	19
venous	74.07%	83.33%	78.43%	24

9.3 Confusion matrix

Rows are true classes, columns are predicted classes.

		PREDICTED							total
TRUE		BG	DI	NS	PR	SU	VE		
	BG	[14	0	1	0	0	0]	15	
	DI	[0	16	0	3	0	4]	23	
	NS	[0	1	14	0	0	0]	15	
	PR	[0	2	0	7	4	2]	15	
	SU	[0	2	0	5	11	1]	19	
	VE	[0	0	0	4	0	20]	24	

		14	21	15	19	15	27	111	

9.4 Arithmetic verification

Every number reconciles: diagonal 82, accuracy $82/111 = 73.87\%$, each precision equals diagonal over column sum, each recall equals diagonal over row sum, macro F1 = 74.49%, weighted F1 = 74.33%. This confirms the evaluation output is internally coherent; split cleanliness is proven by the audit (Section 12.2).

10. Interpretation

10.1 Class difficulty tiers

Tier	Classes	F1 range	Reading
Solved	background, normal-skin	0.93 to 0.97	Reliable on this test set
Working	venous, diabetic	0.73 to 0.78	Usable with caution
Weak	surgical	0.65	Not dependable
Weak	pressure	0.41	Not usable for any decision

The two non wound classes are structurally distinct from wound tissue, so their high scores are expected, and they lift the headline number.

10.2 Wound only performance

Restricted to the 81 true wound images, the six class system gets 54 correct: **66.67%**. This is a derived view of the same matrix, not a separately trained model, and it is the more clinically meaningful figure.

10.3 Pressure behaves as a confusion sink

The model assigned 19 images to pressure; only 7 were correct. The other 12 came from surgical (5), venous (4) and diabetic (3). Meanwhile 8 of 15 true pressure images escaped, mainly to surgical (4) and venous (2). Low precision and low recall together mean the backbones have not learned a discriminative representation for pressure, and the gate cannot recover a signal no backbone provides. A pressure prediction is currently correct roughly one time in three and should not be acted on.

10.4 The pressure and surgical pair

Pressure to surgical: 4. Surgical to pressure: 5. Together 9 of 29 errors, about 31% of all mistakes. Possible explanations (visual similarity of granulating wounds, lost body location context in ROI crops, small per class samples) are hypotheses, not proven causes.

10.5 Other observations

- Diabetic errors flow mainly to pressure (3) and venous (4); both are lower limb chronic ulcers separated clinically by location and history.
- Venous recall is strong (83.33%); its precision drops mainly because four venous images were called pressure.
- The single background to normal-skin confusion is clinically harmless.

11. Statistical Uncertainty

Wilson score intervals, 95% confidence (computed and cross checked):

Quantity	Estimate	Interval
Overall accuracy (82/111)	73.87%	65.0% to 81.1%
Recall, background (14/15)	93.33%	70.2% to 98.8%
Recall, diabetic (16/23)	69.57%	49.1% to 84.4%
Recall, normal-skin (14/15)	93.33%	70.2% to 98.8%
Recall, pressure (7/15)	46.67%	24.8% to 69.9%
Recall, surgical (11/19)	57.89%	36.3% to 76.9%
Recall, venous (20/24)	83.33%	64.1% to 93.3%

One test image moves overall accuracy by about 0.9 points and pressure recall by 6.7 points. Differences under roughly 5 accuracy points between model variants are within noise at this test size. The intervals treat images as independent; patient correlation and label noise are not modelled.

12. Verification Artifacts

12.1 Ablation and fusion comparison (captured via `scripts/make_ablation_table.py`)

All methods are evaluated on the same clean 111 image test set, using the saved fold averaged backbone probabilities (`gating_test_probs_v1.npy`) and the gate's saved predictions (`gating_test_preds_v1.csv`). No retraining was involved. Majority vote ties (9 of 111 images had no two vote majority) were broken by the highest summed probability.

Method	Accuracy	Macro F1	Weighted F1
VGG19 (single)	71.17%	71.09	70.67
DenseNet201 (single)	72.07%	72.80	72.32
MobileNetV2 (single)	74.77%	74.40	74.49
Simple average fusion	72.97%	72.70	72.34
Majority vote (tie to summed probability)	75.68%	75.77	75.44
Gated MLP fusion (final method)	73.87%	74.49	74.33

Reading, stated honestly:

1. **The learned gate clearly outperforms simple probability averaging:** +1.79 macro F1 and +0.90 accuracy. The per image weighting learned from OOF predictions adds measurable value over a fixed one third weighting.
2. **The gate is statistically tied with the strongest single backbone** (MobileNetV2): +0.08 macro F1, -0.90 accuracy.
3. **Majority vote nominally leads by 1.28 macro F1 points**, which is inside the noise band of this test size (Section 11); no method except the gate over averaging is separated beyond noise.

Recommended report wording: *"Gated fusion clearly outperforms simple averaging and matches the strongest individual backbone. Its difference from majority voting is within the statistical uncertainty of the 111 image test set. The gating network is retained as the final method because it consistently beats fixed-weight fusion and, unlike voting, exposes per-sample backbone reliability."*

An honest side observation: on this dataset size, the best single backbone is already strong; the ensemble's measurable benefit is over naive averaging, and the choice between gate and majority vote is not statistically resolvable here.

12.2 Leakage audit (captured, read only runs)

```
python leakage_audit.py :
```

```
Auditing data\processed ...
```

```
=====
SUMMARY
=====
```

```
Exact filename collisions (train vs val/test): 0
  of which BYTE-IDENTICAL (confirmed leak):    0
Group-id collisions (original/aug siblings split across sets): 0

>>> No leakage detected. Your val/test numbers look trustworthy.
```

python scripts/check_clean_split.py :

```
=====
SUMMARY
=====
Exact overlaps across splits: 0
Byte-identical duplicates across splits: 0
GOOD: no exact byte-identical overlap detected.
```

Scope statement: these audits rule out exact filename duplicates, byte identical copies, and augmented sibling groups spanning splits. They do not test near duplicate photographs or same-patient correlation, which remain documented limitations (Section 7.2, Section 13).

12.3 Provenance notes

- **Hardware:** CPU only. The training machine has no NVIDIA GPU (`nvidia-smi` not present, and the training code falls back to CPU when CUDA is unavailable). The runtime pattern is consistent with CPU training: VGG19 (large fully connected head) dominated at roughly 24 hours for 5 folds, while DenseNet201 and MobileNetV2 each finished in roughly 3 to 4 hours.
- **Approximate total runtime:** 30+ hours of backbone training, plus a few minutes for the gating MLP (tabular inputs) and evaluation. This also explains the recorded Phase 1 only policy: Phase 2 would have roughly doubled an already long CPU run.
- Per fold epoch counts were console only; documented as "up to 8 epochs with early stopping patience 5" (Section 8.1).
- Training was performed locally rather than on cloud GPU services, which strengthens reproducibility (no hidden service state) and kept the entire pipeline on one machine.

12.4 Repository fixes included

1. `scripts/gen_oof_preds.py` line 29 was `import #type: ignore` (SyntaxError, script unrunnable in the committed state). Fixed to `import torch # type: ignore`.
2. Added `scripts/make_ablation_table.py` — reproduces Section 12.1 from saved artifacts without re-training.
3. Recommended housekeeping: update `configs/default.yaml` to the real run settings or mark it legacy, and reference `GatingMLP` (in `scripts/fusion/train_gating_mlp.py`) as the canonical gate in the README.

13. Limitations

1. Test set size: 111 images, 15 to 24 per class, wide confidence intervals (Section 11).
2. Single dataset (AZH); other cameras, centres and populations untested.
3. No patient level guarantee: grouping is by source stem; patient identifiers are not part of the ROI release.
4. Pressure and surgical are not dependable (Section 10); the system is a research prototype, not a clinical tool.
5. Gate input shift: trained on single model OOF probabilities, evaluated on fold averaged probabilities (Section 8.3).
6. Per fold epoch counts and gate stopping epoch were printed to console only and not persisted; documented as "up to N with early stopping".
7. Single final evaluation: any future tuning must use train/val/OOF signals only, never the test set.
8. On this test size, the gate's advantage does not extend beyond simple averaging to a statistically clear win over majority voting (Section 12.1); this is reported rather than hidden.

14. Conclusions

The project delivers a correctly designed three backbone fusion system with a learned gating network, rebuilt on a leakage audited, group aware data foundation. Its first clean iteration achieved 73.87% accuracy and 74.49% macro F1; a disciplined improvement round (longer training, Phase 2 fine tuning, label smoothing), selected on out of fold metrics and evaluated once on the same clean test set, raised this to **79.28% accuracy and 79.43% macro F1**. In the final configuration the learned gate outperforms single backbones, simple averaging and majority voting, providing direct experimental support for the gated fusion thesis.

The most significant result of this project is not any single accuracy figure. The team audited its own pipeline, found two leakage bugs and a non group aware split, archived the inflated results, reported the lower honest number, and then improved that honest number through a controlled, leakage safe iteration.

15. Future Work (priority order)

1. Pressure class remains the unresolved weakness (F1 0.44 in the improved run): more pressure data is the most direct remedy.
2. Class weighted or focal loss experiments for the pressure and surgical boundary (selected on OOF/validation only).
3. Gate calibration (temperature scaling, selected on validation only).
4. External dataset validation before any clinical claim.
5. Patient level grouping if identifiers ever become available.

16. Improvement Round (Second Iteration)

After the baseline was frozen, a single improvement round was executed on a free cloud GPU (Google Colab, NVIDIA T4). The data pipeline, splits and test protocol were unchanged; only the training recipe was extended.

16.1 Recipe changes

Element	Baseline	Improvement Round	Rationale
Phase 1 epochs	up to 8	up to 20 (early stopping patience 5)	Baseline logs showed validation loss still falling at epoch 8
Phase 2 fine tuning	not executed	executed (last feature block, lr/10)	The designed improvement lever that CPU constraints had prevented
Label smoothing	0.0	0.1	Reduces overconfidence; suits visually overlapping wound classes
Hardware	local CPU (~30 h)	cloud GPU T4 (~5 h total)	Made the full recipe affordable

Class weighting was deliberately **not** used: training data is already balanced at 350 images per class after augmentation.

16.2 Evaluation discipline

The decision to run the final test evaluation was made **only after** out of fold comparison showed consistent gains. Test labels were not consulted during recipe selection. The improved system was evaluated on the clean test set exactly once, using the same protocol as the baseline (fold checkpoint averaging per backbone, gate inference). The identical configuration, re-run after a session failure, reproduced identical loss values, confirming deterministic seeding.

16.3 Out of fold comparison (decision basis)

Model	Baseline OOF macro F1	Improved OOF macro F1	Change
VGG19	73.23	73.25	+0.01
DenseNet201	67.51	70.32	+2.81
MobileNetV2	66.92	71.95	+5.03

The recipe lifted the two weaker backbones substantially while leaving the strongest single unchanged, narrowing the ensemble's internal spread.

16.4 Improved test results (n = 111)

Metric	Baseline	Improved	Change
Accuracy	73.87% (82/111)	79.28% (88/111)	+5.41
Macro F1	74.49%	79.43%	+4.94
Weighted F1	74.33%	79.44%	+5.11
95% Wilson interval (accuracy)	65.0% to 81.1%	70.8% to 85.8%	shifted upward; intervals overlap

Per class recall (baseline to improved): background 93.3% to **100%**, normal-skin 93.3% to **100%**, diabetic 69.6% to **82.6%**, surgical 57.9% to **63.2%** (precision 73.3% to 75.0%), venous 83.3% unchanged (precision 74.1% to 80.0%), pressure 46.7% unchanged.

Improved confusion matrix (rows = true, columns = predicted):

```
[[15  0  0  0  0  0]
 [ 0 19  0  1  0  3]
 [ 0  0 15  0  0  0]
 [ 0  2  0  7  4  2]
 [ 0  2  0  5 12  0]
 [ 0  0  0  4  0 20]]
```

Notable structural changes: surgical-to-pressure errors fell from 5 to 0, both non-wound classes became perfect, and diabetic errors fell from 7 to 4. The pressure class remains the unresolved weakness (Section 10.3 stands).

16.5 Improved ablation — the gate now wins every comparison

Method	Accuracy	Macro F1	Weighted F1
VGG19 (single)	71.17%	71.31	70.69
DenseNet201 (single)	76.58%	76.45	76.45
MobileNetV2 (single)	73.87%	72.71	73.01
Simple average fusion	78.38%	77.70	78.04
Majority vote	78.38%	77.66	77.77
Gated MLP fusion (final method)	79.28%	79.43	79.44

Gate advantages: +2.98 macro F1 over the best single backbone, +1.72 over simple averaging, +1.76 over majority voting. In the baseline round the gate was statistically tied with the strongest single model and nominally behind majority voting; with improved backbones the learned gate outperforms every alternative. This is direct experimental evidence for the central thesis claim: per-image learned weighting adds measurable value over fixed fusion strategies, and the value grows as backbone quality improves.

16.6 Reporting note

Both results are reported. The baseline (73.87%) documents the clean pipeline's first honest iteration; the improved system (79.28%) is the final method. Neither number has been tuned against the test set: recipe selection used OOF metrics only, and each configuration was evaluated on the test set exactly once. Given $n = 111$, the accuracy improvement (+5.4 points) is substantial and directionally consistent with the OOF gains, though the confidence intervals overlap; the claim is stated with this uncertainty attached.

Appendix A. Reference Command Sequence (verified against README history)

```
# 1. Rebuild clean split (seed 42, 70/15/15)
python scripts/rebuild_clean_split.py

# 2. Augment training folder only (350 per class, seed 42)
python scripts/augment_train_only.py

# 3. Audit -> PASSED, see Section 12.2
python leakage_audit.py
python scripts/check_clean_split.py

# 4. OOF per backbone (Phase 1 only; add --phase2 for the future improvement run)
python scripts/gen_oof_preds.py --model vgg19 --folds 5 --phase1-epochs 8 --batch-size 16 --
lr 1e-4 --weight-decay 1e-4 --output outputs/01_oof/vgg19_clean
python scripts/gen_oof_preds.py --model densenet201 --folds 5 --phase1-epochs 8 --batch-size 16 --
lr 1e-4 --weight-decay 1e-4 --output outputs/01_oof/densenet201_clean
python scripts/gen_oof_preds.py --model mobilenet_v2 --folds 5 --phase1-epochs 8 --batch-size 16 --
lr 1e-4 --weight-decay 1e-4 --output outputs/01_oof/mobilenetv2_clean

# 5. Gate on OOF + clean test evaluation
python scripts/fusion/train_gating_mlp.py --models vgg19,densenet201,mobilenet_v2 --oof-dir out-
puts/01_oof --output outputs/02_gating

# 6. Final metrics
python scripts/evaluate_gated_fusion.py --preds-csv outputs/02_gating/gating_test_preds_v1.csv --
output outputs/03_figures

# 7. Ablation table (no retraining) -> Section 12.1
python scripts/make_ablation_table.py
```

Note: the commands in step 4 are the ones documented in the repository history for the reported run; they contain no `--phase2` flag, which is the basis of the Phase 2 conclusion in Section 8.1.

Appendix B. Plain Language Glossary

Term	Plain meaning
Sample	One image
Label	The correct category of an image
Epoch	One full pass through a run's training data
Fold	One partition used in cross validation
Group	One source image plus all its augmented copies; they stay on one side of a split
OOF prediction	A prediction made by a model that never trained on that sample's group
Backbone	The large pretrained feature extracting network
Head	The final layer that turns features into class scores
Frozen	Parameters kept unchanged during training
Fine tuning	Unfreezing part of the backbone and training it gently at a lower learning rate
Gating network	The learned combiner of the three backbones
Data leakage	Evaluation data influencing training or selection in an invalid way
Precision	Of the items predicted as class X, the fraction that were truly X
Recall	Of the items truly in class X, the fraction the model found
F1	The balance of precision and recall
Macro F1	F1 averaged equally across classes
Weighted F1	F1 averaged with class sizes taken into account
Confidence interval	The range where the true value plausibly lies given the sample size
Ablation study	Removing or replacing parts of a system to measure what each part contributes

Appendix C. Status Scoreboard

Item	Status
Leakage bugs identified, fixed and documented	Done, verified in code
Group aware splitting (StratifiedGroupKFold, stem groups)	Done, verified in code
Clean split rebuilt from raw data, seed 42, 70/15/15	Done, verified in code + counts (7.3)
Contaminated artifacts archived and excluded	Done (archive/cleanup_2026_09_12/ , local)
Backbones trained, Phase 1 only, 5 fold group CV	Done, verified (commands + script)
Gating MLP trained on OOF, validated on OOF split only	Done, verified in code
Clean test evaluation, single pass	Done, results file in repo
Metric arithmetic independently rechecked	Done (Section 9)
Per class analysis, confusion sink, wound only view	Done (Section 10)
Confidence intervals	Done (Section 11)
Phase 2 status resolved	Done: baseline = did not run; executed in Improvement Round (Sections 8.1, 16)
gen_oof_preds.py import bug found and fixed	Done; commit with report update
Ablation tables (baseline + improved)	Done, captured (Sections 12.1, 16.5)
Leakage audit output	Done, captured, PASS (Section 12.2)
Split counts (all classes, all splits)	Done, captured (Section 7.3)
GPU model and runtime recorded	Done: CPU only, ~30+ h total (Section 12.3)
Before/after leakage numbers (optional)	Optional, from local archive
Improvement Round executed	Done: 79.28% accuracy, 79.43% macro F1 (Section 16)
e work, pending supervisor approval	